AI INNOVATION HACKATHON 2026 — ROUND 2 TECHNICAL ASSESSMENT

ShikshakAI (शिक्षक AI)

The Adaptive Human-Like AI Educator That Teaches Through Video

ShikshakAI transforms textbooks, notes, research papers, presentations, or direct topic prompts into a structured, personalized, audiovisual classroom. Featuring animated, lip-synced AI teacher personas, a subject-aware smartboard (KaTeX derivations, interactive code sandboxes, 3D flashcards), real-time misconception diagnosis, and personalized 7-day revision roadmaps, ShikshakAI emulates the cognitive behavior of a master 1-on-1 human educator at a 100% zero operational cost.

Project Attribute	Technical Specification
Challenge Title	AI Teacher: Build a Human-Like AI Educator That Teaches Through Video
Full-Stack Architecture	FastAPI (Python 3.10+) Backend + React 19 / Vite 8 Frontend Stage
Primary AI Brain	Google Gemini 2.0 Flash (google-genai SDK) with JSON Schema Constraints
Secondary / Critic AI	Groq LPUs (Qwen 3.6 / Llama 3.3 / GPT-OSS) for Real-Time Critic QA
Knowledge Grounding (RAG)	PyPDF, python-docx, python-pptx extraction + ChromaDB Vector Grounding
Neural Speech Synthesis	Microsoft Edge-TTS (Multi-Voice, In-Memory Base64 MP3 Audio Streaming)
Avatar & Lip-Sync Engine	Vector SVG Teacher Models with Web Audio API 60 FPS Direct-DOM Lip-Sync
Interactive Smartboard	KaTeX Equations, Live Code Runner, 3D Flashcards, Mermaid Mindmaps
Multilingual Coverage	English (en-IN / en-US), Hindi (हिंदी), and Hinglish (Code-Switching)
Operating Cost	$0.00 / month (100% Zero-Cost Open Production Stack Architecture)
Track Leads & Team	Kushal Singh (Track A: Backend AI Brain) & Jatin Rawat (Track B: Frontend Classroom)

ZERO-BUDGET ARCHITECTURE: Zero-Cost Production Guarantee: ShikshakAI delivers industrial-grade AI performance with $0 cloud bills by utilizing Gemini 2.0 Flash's free tier, Microsoft Edge-TTS's unlimited neural voices, client-side Web Speech recognition, local ChromaDB embeddings, and browser-native MediaRecorder canvas capture.

Table of Contents

1. Problem Statement

Modern digital learning remains polarized into two distinct paradigms that each fail to replicate the effectiveness of a master human educator:

1. Pre-Recorded Static Video Lectures (Coursera, YouTube, MOOCs):
   - Non-Adaptive: Delivered at a uniform, rigid pace regardless of learner confusion or prior knowledge.
   - Zero Active Feedback: Cannot determine whether a student understands a derivation or harbors subtle misconceptions.
   - Passive Consumption: Encourages superficial watching without interactive derivation stepping or code execution.
   - High Attrition: Global completion rates hover below 8–10% across MOOC platforms.
2. Generic Text-Only Chatbots (ChatGPT, Claude, Q&A; Assistants):
   - Text Walls: Overwhelms students with unbroken blocks of text lacking pedagogical pacing or vocal tone.
   - Absence of Visual Grounding: Fails to provide synchronized formulas, live code sandboxes, diagrams, or memory mnemonics.
   - Lack of Presence & Empathy: No eye contact, expressive gesturing, or reassuring neural voice modulation.
   - Passive Q&A; Loop: Relies entirely on the student to identify what they do not know and ask the right questions.

The Socioeconomic Tutoring Gap: Bloom's 2 Sigma Problem demonstrated that 1-on-1 personalized tutoring elevates student achievement by two standard deviations (over 98% of classroom peers). Yet high-quality human tutors cost $30–$100 per hour, making personalized learning a privilege for the few. ShikshakAI democratizes masterclass 1-on-1 virtual education at zero cost.

2. Solution Overview

ShikshakAI (शिक्षक AI) is an autonomous, adaptive virtual classroom where an AI teacher instructs students through synchronized neural speech, dynamic blackboard visuals, interactive checkpoints, and lip-synced teacher avatars.

Rather than operating as a simple query-response chatbot, ShikshakAI implements a deterministic pedagogical state machine spanning 8 interconnected phases:

Stage	Phase Name	Pedagogical Functionality & Technical Implementation
1	Understand	Ingests textbooks, PDFs, PPTX slides, notes, or topic queries via pypdf / python-docx / ChromaDB vector search to extract core concepts.
2	Plan	Generates a structured visual teaching storyboard with clear scene objectives, durations, visual component types, and checkpoints.
3	Explain	Synthesizes natural spoken explanations via Edge-TTS with persona-matched inflection in English, Hindi, or Hinglish.
4	Demonstrate	Renders synchronized smartboard visuals including step-by-step KaTeX derivations, live code runners, 3D flashcards, and Mermaid mindmaps.
5	Question	Engages the student mid-lesson through Socratic checkpoints requiring multiple-choice, text, or voice responses.
6	Evaluate	Diagnoses student answers via LLM pedagogical critic to identify not just correctness, but the exact conceptual misconception.
7	Adapt	Dynamically generates and splices remedial scenes featuring physical balance analogies, counter-examples, or alternative visual breakdowns.
8	Continue	Advances through the lesson, updates real-time analytics, generates a 7-day spaced repetition revision roadmap, and exports WebM video.

3. Key Features

3.1 Multiple Expressive Teacher Personas:

• Dr. Maya (Physics & Mathematics Specialist): Analytical, structured, Socratic teaching style; styled in a formal navy blazer, sleek glasses, and professional demeanor.
• Prof. Alex (Computer Science & AI Engineer): Practical, code-focused, energetic approach; styled with a dark hoodie, headset, and technical focus.
• Ananya Ma'am (Humanities, Literature & Hindi Mentor): Empathetic, warm, narrative storytelling tone; styled in traditional attire with expressive pacing.
• Acoustic Lip-Sync & Blinking: Direct-DOM frequency-driven mouth animation at 60 FPS synchronized to audio syllables, with natural 4.2-second eye-blinking intervals.

3.2 Subject-Aware Interactive Smartboard:

• Step-by-Step KaTeX Derivations: Mathematical breakdown with sequential step progress, formula annotations, and interactive sliders.
• Interactive Code Sandbox Runner: In-browser code runner featuring syntax highlighting, line numbers, live simulated execution terminal, status output, and variable watch.
• 3D Interactive Revision Flashcards: Click-to-flip 3D cards with memory mnemonics, front/back perspectives, and mastery tracking ('Mark as Understood').
• Mermaid Concept Mindmaps: Instant workflow diagrams, decision trees, and hierarchical relationship maps.
• Specialized Visual Domain Renderers: Balance beams for algebraic equivalence, physics circuit/gravity simulations, and biochemical reaction flows.

3.3 Misconception Detection & Active Dynamic Adaptation:

• Multi-Modal Student Input: Students respond via click selection, typed natural language, or voice input (Web Speech API).
• Pedagogical Diagnostician: Detects faulty mental models (e.g., confusing current with resistance).
• Dynamic Remediation Splicing: Automatically generates and splices a targeted remediation scene with physical analogies before advancing.

3.4 Learning Analytics, 7-Day Spaced Repetition & Video Export:

- Radial Mastery Gauge: Real-time accuracy gauge tracking correct responses and conceptual progress.
  - 7-Day Retention Roadmap: Day-by-day revision milestones (Recall, Derivation, Misconception Busting, Code Drills, Interleaving, Timed Drill, Feynman Check).
  - In-Browser Video Recording: MediaRecorder API captures visuals, avatar animations, and neural audio into downloadable .webm video.

4. System Architecture

ShikshakAI operates as a cohesive full-stack decoupled architecture optimized for sub-second responsiveness, zero operational cost, and cloud/serverless readiness.

Subsystem Layer	Primary Technologies	Architectural Responsibilities
Frontend Stage (Track B)	React 19, Vite 8, Vanilla CSS3, Lucide-React	Virtual classroom UI, stage management, stateful timeline stepper, navigation views.
Avatar & Visuals	SVG Vector Engine, Web Audio API, KaTeX, Mermaid.js	60 FPS Direct-DOM lip-sync, LaTeX formula rendering, interactive code sandbox, 3D card transforms.
Backend AI Brain (Track A)	FastAPI, Uvicorn, Pydantic V2, aiofiles	Asynchronous REST router, pedagogical state engine, JSON schema validation, in-memory session caching.
AI Storyboard & LLMs	Google Gemini 2.0 Flash, Groq LPUs, google-genai	Visual storyboard planning, AI critic validation pass, student response diagnosis, adaptive scene generation.
RAG Knowledge Store	pypdf, python-docx, python-pptx, ChromaDB	Document extraction, chunking, slide indexing, semantic retrieval, and grounding prompt injection.
Speech & Neural TTS	Microsoft Edge-TTS, Web Speech API	Regex spoken prose pre-processing, in-memory Base64 MP3 audio streaming, client-side voice input.
Media & Capture	HTML5 MediaRecorder API, Canvas Audio Loop	In-browser screen and audio capture, client-side WebM video export without cloud GPU rendering.

5. AI/ML Models Used

ShikshakAI incorporates a multi-tiered ensemble of state-of-the-art AI/ML models:

5.1 Primary Pedagogical LLM: Google Gemini 2.0 Flash

Accessed via the official google-genai Python SDK. Configured with response_mime_type='application/json' and temperature 0.4. Responsible for transforming raw topic queries and extracted textbook chapters into strictly validated visual teaching storyboards, conducting misconception diagnostics, and generating adaptive remediation scenes.

5.2 Secondary / AI Critic LLM: Groq Hosted Open Weights

Powered by ultra-fast Groq LPUs executing qwen/qwen3.6-27b, llama-3.3-70b-versatile, and openai/gpt-oss-20b. Serves two critical functions: (1) Running a pedagogical critic pass that inspects generated storyboards for factual accuracy and visual density, and (2) Acting as an automatic fallback LLM when Gemini API quotas are exhausted.

5.3 Neural Acoustic Speech Synthesizers: Microsoft Edge-TTS

Executes Microsoft's cutting-edge neural TTS models (e.g., en-IN-NeerjaNeural, en-IN-PrabhatNeural, hi-IN-SwaraNeural, hi-IN-MadhurNeural, hi-IN-KavyaNeural). Provides lifelike prosody, natural breathing pauses, and accurate Indian accent inflections at zero cost.

5.4 Browser Automatic Speech Recognition (ASR): Web Speech API

Client-side speech-to-text inference running locally on the user's browser (webkitSpeechRecognition). Supports real-time transcription in English (en-IN) and Hindi (hi-IN) with zero server latency and zero API cost.

5.5 Intelligent Subject-Aware Fallback Synthesizer

A deterministic, rule-based algorithmic synthesizer that generates rich, subject-aware lesson plans across Linear Algebra, Physics (Ohm's Law, Hooke's Law), Computer Science (Binary Search, C Programming), and Biology (Photosynthesis) ensuring 100% platform availability even when completely offline or unkeyed.

6. RAG (Retrieval-Augmented Generation) Implementation

ShikshakAI's RAG pipeline enables students to upload real-world educational resources and receive accurate, grounded virtual lessons without hallucination:

1. Multi-Format Ingestion Pipeline:
   • PDF Extraction: Uses pypdf.PdfReader to extract text page-by-page with page-boundary metadata.
   • Word Extraction: Uses docx.Document to extract paragraph text, headings, and assignments.
   • PowerPoint Slide Extraction: Uses pptx.Presentation to parse slide shape texts and deck structure.
   • Plain Text: Direct UTF-8 ingestion with fallback character encoding tolerance.
2. Vector Indexing & Active Grounding:
   • Ingested content is indexed into a local ChromaDB vector store and cached in the active session map (ACTIVE_DOCUMENTS).
   • When a lesson plan is requested, up to 6,000 characters of extracted textbook material are injected directly into the planning prompt.
   • The LLM is strictly instructed: 'Ground your visual lesson in this source textbook material... do not introduce contradictory external concepts.'

GROUNDED RAG COMPLIANCE: Hallucination Minimization: By anchoring the visual director prompt to extracted document text and enforcing verifiable definitions, formulas, and diagrams, ShikshakAI delivers strictly syllabus-compliant instruction.

7. Prompt & Agent Architecture

The AI Brain is structured around modular, specialized prompt agents that collaborate to choreograph lessons:

7.1 Master Visual Director Agent (STORYBOARD_SYSTEM_PROMPT):

Enforces the fundamental visual-first philosophy: 'Your job is NOT to write an essay or a ChatGPT chat answer. Your job is to design a VISUAL TEACHING STORY where the student understands the concept primarily by WATCHING.'

- Component Catalog: Enforces mapping each scene to one valid component: ConceptReveal, StepByStep, EquationBuild, CodeExecution, DataStructure, ProcessFlow, Comparison, Molecule, Anatomy, Summary.
  • Visual Hierarchy: Enforces 1 primary objective per scene and minimal on-screen text.
  • Narration Purity: Prohibits raw Markdown symbols and LaTeX code in spoken narration strings.

7.2 Validator & Auto-Repair Engine:

The validate_and_repair_storyboard() function guarantees that even malformed LLM outputs are repaired deterministically: normalizes difficulty, infers missing subjects, repairs component names, removes illegal narration characters, and ensures non-empty visual element coordinates.

7.3 AI Critic Pass (run_ai_critic_pass):

A secondary inspection agent executes over Groq LPUs. It verifies factual accuracy, confirms that text has been converted into visual structures, validates component choices, and verifies narration purity within a strict 6-second timeout window.

7.4 Pedagogical Misconception Diagnostician Agent:

The evaluate_student_response() agent receives the question, correct answer, misconception guide, and student answer. It diagnoses whether the answer is conceptually sound, identifies the specific misconception, provides encouraging feedback with a fresh analogy, and emits an adaptive_action ('proceed' or 're_explain_with_analogy').

7.5 Contextual In-Studio Doubt Agent (ask_contextual_teacher):

Accepts live student questions during active scenes, conditioned on the active scene title and on-screen visual payload, returning concise, context-aware answers delivered strictly in the selected teacher persona.

8. Personalization Approach

ShikshakAI adapts dynamically to each learner across multiple pedagogical dimensions:

Dimension	Options / Range	Pedagogical Adaptation Mechanism
Learner Level	Beginner, Intermediate, Advanced	Beginner: Simple analogies, foundational terminology. Advanced: Mathematical derivations, code implementations, formal edge cases.
Time Budget	5 mins, 20 mins, 60 mins	5 min: High-yield concept summary. 20 min: Standard 6-scene lesson with checkpoint. 60 min: Comprehensive multi-step derivations and labs.
Teacher Persona	Dr. Maya, Prof. Alex, Ananya Ma'am	Dr. Maya: Structured, analytical Socratic. Prof. Alex: Practical, code-focused, energetic. Ananya Ma'am: Warm, empathetic storytelling.
Language Preference	English, Hindi (हिंदी), Hinglish	Translates prompts, explanations, checkpoints, and routes to authentic native neural voice accents.
Dynamic Branching	Real-Time Misconception Splicing	When a student fails a checkpoint, an adaptive remediation scene (e.g. balance scale analogy) is spliced immediately into the lesson graph.

9. Assessment Methodology

ShikshakAI rejects punitive, superficial testing in favor of formative, diagnostic cognitive evaluation:

1. Formative Mid-Lesson Checkpoints: Checkpoints appear at critical conceptual milestones, pausing narration and requiring student engagement before unlocking subsequent derivation steps.
2. Multi-Modal Input: Students can answer via multiple-choice click, typed natural language, or voice input via the Web Speech API.
3. Semantic Diagnostic Evaluation: The evaluation engine does not perform naive string matching. It evaluates conceptual correctness and identifies the exact underlying misconception.
4. Dynamic Assessments Hub (AssessmentsView.jsx): Automatically generates customized quizzes from the student's past searched concepts, tracking topic retention scores and offering a 'Grand Review Assessment'.
5. 7-Day Spaced Repetition Roadmap (LearningReportModal.jsx): Delivers a research-backed 7-day study plan covering formula recall, visual derivation, misconception busting, code drills, interleaving, speed drills, and the Feynman technique.

10. Multilingual Implementation

ShikshakAI provides native, end-to-end multilingual instruction across English, Hindi (हिंदी), and Hinglish:

Language Mode	Spoken Voice Model	Pedagogical Scripting & Interaction Style
English (en)	en-IN-NeerjaNeural / en-IN-PrabhatNeural	Clear, globally accessible Indian English with authentic technical terminology.
Hindi (hi)	hi-IN-SwaraNeural / hi-IN-MadhurNeural / hi-IN-KavyaNeural	Pure Hindi script and vocabulary for conceptual definitions, visual titles, and feedback.
Hinglish (hinglish)	hi-IN-SwaraNeural / hi-IN-MadhurNeural	Conversational code-switching blending Hindi sentence structure with English technical terms.

11. Voice Implementation & Neural Speech

A major technical hurdle in automated AI education is that TTS engines read mathematical symbols and Markdown tokens literally (e.g., reading \frac{1}{2} as 'backslash f-r-a-c open brace one close brace'). ShikshakAI overcomes this via an advanced verbal pre-processing engine:

Spoken Prose Pre-Processing (clean_text_for_speech):

- Converts LaTeX fractions: \frac{a}{b} → 'a over b'
  - Converts square roots: \sqrt{x} → 'square root of x'
  - Converts exponents: x^2 → 'x squared', x^3 → 'x cubed', x^n → 'x to the power of n'
  - Converts chemical notations: CO_2 → 'CO 2', H_2O → 'H 2O'
  - Converts mathematical operators: \cdot → 'times', \approx → 'approximately', \neq → 'is not equal to', \pm → 'plus or minus'
  - Strips all Markdown formatting (*, _, #, `) and code fences.
  - Cleanses punctuation pauses to produce fluid, human-like cadence.

In-Memory Base64 Audio Streaming: Audio synthesized via edge-tts is converted directly into in-memory Base64 MP3 Data URIs (data:audio/mp3;base64,...). This eliminates server disk I/O bottlenecks and enables instant playback across serverless environments.

12. Avatar & Video Generation Approach

12.1 SVG Vector Avatar Architecture:

Avatars are implemented as responsive, scalable SVG illustrations featuring layered radial lighting gradients, studio camera framing grids, hair geometries, facial expressions, and clothing specific to each teacher persona (Dr. Maya's navy blazer; Prof. Alex's tech hoodie; Ananya Ma'am's traditional attire).

12.2 Real-Time Acoustic Lip-Sync (60 FPS Direct-DOM):

Rather than using heavy cloud video rendering, ShikshakAI analyzes speech audio in real time on the client:

1. Attaches a Web Audio API AudioContext and AnalyserNode (fftSize: 64, smoothingTimeConstant: 0.6) to the active audio stream.
2. A requestAnimationFrame loop samples frequency bins to calculate instantaneous loudness and speech formants.
3. Direct DOM Ref Mutation: The loop mutates the SVG mouth element attributes (rx, ry, opacity) directly on the DOM ref, achieving butter-smooth 60 FPS lip synchronization with 0% React re-rendering overhead.
4. Autonomous eye-blinking cycles run every 4.2 seconds to maintain lifelike presence.

12.3 In-Browser Video Recording & Export (LessonRecorder.jsx):

Utilizes the native HTML5 MediaRecorder API and navigator.mediaDevices.getDisplayMedia with tab audio loopback. Captures live classroom visuals, avatar animations, blackboard updates, and audio into high-definition .webm video files ready for offline viewing without expensive cloud GPU render farms.

13. APIs and Third-Party Services

Endpoint	Method	Payload / Parameters	Description
/api/health	GET	None	Verifies health connectivity and operational zero-cost stack.
/api/config	GET	None	Checks active configured AI engine (Gemini, Groq, or Local).
/api/config/key	POST	gemini_api_key, groq_api_key	Dynamically updates and persists API keys in .env.
/api/upload	POST	multipart/form-data file	Ingests PDF, DOCX, PPTX, TXT for RAG knowledge grounding.
/api/lesson/plan	POST	topic, level, duration, language	Generates structured visual teaching storyboard.
/api/tts/speak	POST	text, language, teacher_id	Synthesizes neural voice audio via Microsoft Edge-TTS.
/api/evaluate	POST	question, student_answer, correct	Diagnoses student answer and returns adaptive remediation.
/api/studio/ask	POST	topic, scene_title, question	Context-aware teacher answering live doubts during an active scene.
/api/studio/adapt	POST	topic, misconception, question	Generates targeted adaptive remediation scene with alternate analogy.
/api/course/generate	POST	topic, learner_level, language	Generates full comprehensive multi-module course curriculum.

14. Setup & Local Installation Instructions

Follow these steps to configure and launch ShikshakAI in a local development environment:

```
# 1. Clone the repository
git clone https://github.com/Kushalsingh1234/ShikshakAI.git
cd ShikshakAI
# 2. Configure & launch the Backend (Track A)
cd backend
python3 -m venv venv
source venv/bin/activate       # On Windows: .\venv\Scripts\activate
pip install -r requirements.txt
# Configure environment variables
cp .env.example .env
# Edit .env and insert your GEMINI_API_KEY (from https://aistudio.google.com/)
uvicorn main:app --reload --port 8000
# 3. Configure & launch the Frontend (Track B) in a new terminal
cd frontend
npm install
npm run dev
# Classroom UI is now running at http://localhost:5173
```

15. Production Deployment Instructions

Frontend Vercel / Netlify Deployment: Configured via root vercel.json to route API calls to the backend and serve the Vite client app.
Backend Container Deployment (Docker / Cloud Run): Packaged using the following production Docker configuration:

```
FROM python:3.11-slim
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends build-essential && rm -rf /var/lib/apt/lists/*
COPY backend/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY backend/ .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "4"]
```

16. Known Limitations & Future Roadmap

16.1 Current Technical Limitations:

• Edge-TTS Outbound Internet: Microsoft Edge-TTS requires an active internet connection to stream neural voices; offline execution relies on standard browser synthesis.
• Speech Recognition Browser Compatibility: Web Speech API is natively optimized for Chromium browsers (Chrome, Edge, Brave); Safari and Firefox have limited support and fall back to text input.
• Scanned Image PDFs: Text extraction currently parses native text layers via pypdf; scanned photocopies without OCR layers require pre-processing.

16.2 Future Technical Roadmap:

• On-Device Whisper & Piper TTS: Integrating WebAssembly-based Whisper ASR and Piper neural TTS for 100% offline, air-gapped speech synthesis.
• Multi-Student Collaborative Classroom: WebRTC-based virtual study rooms where multiple students can participate in shared AI-led group lessons.
• Multimodal Handwriting OCR: Ingesting student notebook photos and whiteboard sketches directly using Gemini Vision models.
• Photorealistic 3D Avatar Rendering: WebGL / Three.js avatars with 3D skeletal rigging and facial action coding (FACS).

CONCLUSION: ShikshakAI demonstrates that cutting-edge, personalized 1-on-1 education is achievable at zero operational cost, combining rigorous pedagogical state machines, real-time multimodal interaction, and empathetic teacher avatars to empower every learner worldwide.